O1 Deep Tech Fact-Check Blueprint (for next Opus 4.6 iteration)

Дата: 2026-02-09
Проект: `TwoComms / dtf.twocomms.shop`
Цель документа: дать следующему исследовательскому агенту полный, технически точный контекст о текущем состоянии проекта, качестве отчета `Ideas/Audit/o1/1.md`, приоритетных внедрениях по `twocomms/Promt/Effects.MD`, и о реальных инфраструктурных ограничениях production.

---

1. Что именно проверено

1.1 База отчетов/идей

Проверены файлы:

- `Ideas/Audit/o1/1.md` (основной целевой отчет Opus)
- `Ideas/Audit/opus 4.6 v1.MD`
- `Ideas/Audit/opus 4.6 v2.MD`
- `Ideas/Audit/FINAL_TECH_AUDIT_FOR_OPUS_4_6_2026-02-09.md`
- `Ideas/Audit/MASTER_TECH_AUDIT_2026-02-09.md`
- `Ideas/Audit/Antigravity_Deep_Audit.md`
- `Ideas/Audit/Antigravity_Gap_Analysis.md`
- `Ideas/Audit/CONSOLIDATED_EFFECTS_GAP_ANALYSIS.md`
- `Ideas/CODEX_Report.md`
- `Ideas/Antigravity_Report.md`
- `Ideas/Gemini_1.5_Pro.md`

1.2 Технические источники проекта

- `twocomms/Promt/Effects.MD`
- `twocomms/dtf/templates/dtf/base.html`
- `twocomms/dtf/templates/dtf/index.html`
- `twocomms/dtf/templates/dtf/order.html`
- `twocomms/dtf/templates/dtf/constructor_app.html`
- `twocomms/dtf/static/dtf/js/components/core.js`
- `twocomms/dtf/static/dtf/js/components/effect.compare.js`
- `twocomms/dtf/static/dtf/js/components/multi-step-loader.js`
- `twocomms/dtf/static/dtf/js/components/vanish-input.js`
- `twocomms/dtf/static/dtf/js/components/effect.infinite-cards.js`
- `twocomms/dtf/static/dtf/js/components/floating-dock.js`
- `twocomms/dtf/static/dtf/js/dtf.js`
- `twocomms/dtf/preflight/engine.py`

- `twocomms/dtf/forms.py`
- `twocomms/dtf/views.py`
- `twocomms/dtf/urls.py`
- `twocomms/twocomms/settings.py`
- `twocomms/twocomms/production_settings.py`
- `twocomms/requirements.txt`

1.3 Production SSH-сверка (факт, а не предположение)

Проверка выполнена по SSH на `2026-02-09`:

- хост: `guru-ua6.hostsila.org`
- `DEBUG=False`
- `CELERY_BROKER_URL` настроен на Redis Cloud URL
- `REDIS_PING` с production завершился ошибкой DNS (`Name or service not known`)
- celery/huey/redis worker-процессы не обнаружены
- cron-задачи активны (в т.ч. `update_tracking_statuses` и `check_survey_inactivity`)

Вывод: инфраструктурно backend сейчас работает как web+cron, не как полноценная worker-архитектура с живым брокером задач.

---

2. На каком этапе проект (текущая фактическая стадия)

Проект уже в рабочем состоянии как продуктовый сайт, но слой “wow-эффектов” и часть preflight UX находятся в смешанном состоянии:

- **работает**: SSR Django, основная структура страниц, формы, базовый constructor flow, sync preflight для конструктора, preflight storage/reporting, эффекты как компонентный слой;
- **частично**: эффектные UI-компоненты реализованы неравномерно (часть зрелые, часть заглушки);
- **проблемные зоны**: конфликт нижнего правого UI, старый/новый JS-подход в одном проекте, несовпадение between report-claims and real code, слабая production-готовность Celery/Redis сценария.

Коротко: это не MVP-черновик и не “нулевой сайт”; это **рабочая продуктовая база с незавершенной унификацией визуального и async-слоя**.

---

3. Факт-чек отчета `Ideas/Audit/o1/1.md`

Ниже оценка ключевых положений `o1/1.md` с точки зрения текущего кода.

Claim из `o1/1.md`	Вердикт	Комментарий по факту
Нужна унификация lifecycle эффектов	`Частично уже сделано`	В `core.js` уже есть `DTF.registerEffect`, init/cleanup и HTMX hooks, но в проекте сохраняются legacy-функции в `dtf.js` и дубли файлов.
Нужны feature flags	`Сделано частично`	Flags уже есть (`DTF_FEATURE_FLAGS` + `get_feature_flags()`), но маппинг флагов на новые эффекты неполный и непоследовательный.
Compare требует доработки (autoplay/hover/keyboard)	`Верно`	Текущий `effect.compare.js` поддерживает range/pointer-drag; нет полноценных autoplay/hover-mode parity, нет расширенной keyboard/a11y логики.
Infinite cards нужно SEO-safe	`Верно`	Текущий `effect.infinite-cards.js` фактически заглушка; нет clone strategy с `aria-hidden/inert`, нет pause-on-hover orchestration.
Multi-step loader сейчас fake	`Верно`	`multi-step-loader.js` делает шаги по таймерам (`setTimeout` 260/520ms), не из реальных backend этапов.
Vanish input недореализован	`Верно`	Сейчас это краткий pulse/shake на invalid; нет циклических placeholder'ов и vanish-clear текста.
FAB/Dock конфликтует	`Верно`	В `base.html` одновременно есть `#dtf-fab`, `data-floating-dock`, и отдельный `.mobile-dock`.
Celery/Redis на shared хостинге проблематичен	`Верно по текущему production`	На production broker DNS не резолвится и worker-процессов нет.
Нужен sync preflight путь	`Верно`	Для текущей инфраструктуры sync+deterministic UX

Claim из `o1/1.md`	Вердикт	Комментарий по факту
		действительно реалистичнее и стабильнее.

3.1 Что в `o1/1.md` нужно скорректировать

1. Формулировка “всё async невозможно” слишком абсолютна. Точная версия: **в текущем production-контуре async через Celery/Redis сейчас недоступен надежно**.
2. Часть рекомендаций “внедрить error boundary” уже реализована в `core.js` через `try/catch` на init каждого эффекта.
3. В отчете местами смешаны контексты страниц:
   - в `constructor_app` preflight уже реальный (sync в форме);
   - в `order` preflight-терминал пока mostly декларативный UI.

---

4. Сверка v1/v2 и других AI-отчетов

4.1 `opus 4.6 v1` и `v2`

- `v1` неполный (обрыв структуры).
- `v2` структурно лучше и пригоден как рабочий каркас.
- Использовать только после fact-check с кодом (этот документ = такой fact-check).

4.2 Практическая ценность источников

- `Ideas/CODEX_Report.md`: высокий инженерный вес, хороший grounded-фундамент.
- `Ideas/Antigravity_Report.md`: сильный креатив-бэклог, ниже точность по архитектурным ограничениям.
- `Ideas/Gemini_1.5_Pro.md`: сильные идеи по behavioral/performance, но часть предложений слишком радикальны для текущего стека.

4.3 Итог по качеству аналитики

Лучший подход: **брать отчеты как гипотезы и фильтровать через код/сервер-факты**.
Этот документ фиксирует такой фильтр и даёт операционный план внедрения.

---

5. Что реально есть в `Effects.MD` и что уже перенесено

`twocomms/Promt/Effects.MD` — это в основном Aceternity React-референсы (CLI `npx shadcn ...`) с демо-кодом.

Ключевые компоненты, которые для нас критичны:

- `Compare`
- `Floating Dock`
- `Infinite Moving Cards`
- `Placeholders And Vanish Input`
- `Sidebar`
- `Multi Step Loader`
- `Text Generate Effect`
- `Images Badge` (для upload badge/иконки в доке)

5.1 Статус переноса в текущем проекте

Компонент	Статус сейчас	Комментарий
Compare	`Частично`	Есть drag/range/pointer, но нет режима “визуально как в референсе” (autoplay/hover orchestration, расширенная a11y).
Floating Dock	`Частично`	Есть минимальный floating nav, но без macOS magnification и с конфликтом через FAB/mobile-dock.
Infinite Cards	`Минимум`	CSS-анимация есть, JS почти пустой, нет SEO-safe cloning.
Vanish Input	`Минимум`	Только invalid pulse/shake; нет vanish-clear/placeholder-cycle.
Multi Step Loader	`Fake`	Ступени переключаются таймерами, не backend-driven.
Sidebar	`Отдельный компонент-референс`	В конструкторе уже есть левый `aside`, но не тот интерактивный эффект expand/sidebar из референса.
Text Generate	`Фрагментарно`	Есть encrypted text эффекты; dedicated text-generate behavior для trust storytelling не оформлен отдельным модулем.

6. Глубокий техразбор must-have внедрений (по вашему ТЗ)

Ниже именно те эффекты/фишки, которые вы обозначили как обязательные.

6.1 Compare “до/после” (визуально как референс)

Цель

Сделать compare визуально максимально близким к референсу из `Effects.MD`:

- поддержка `drag` + `hover` + `autoplay`
- autoplay привлекает внимание, interaction переводит в ручной режим
- mobile fallback без hover

Что уже есть

- `effect.compare.js` уже инициализирует drag/pointer + range
- CSS содержит `touch-action: pan-y` (важно для скролла на mobile)

Что добавить

1. `data-compare-mode="drag|hover|autoplay"`
2. `data-compare-autoplay="true|false"`
3. автопетля 20%→80%→20% с остановкой при user interaction
4. keyboard support (`ArrowLeft/ArrowRight/Home/End`) + ARIA slider semantics
5. reduced-motion disable autoplay

Где интегрировать

- `twocomms/dtf/static/dtf/js/components/effect.compare.js`
- `twocomms/dtf/static/dtf/css/components/effect.compare.css`
- шаблоны с `[data-compare]`:
  - `twocomms/dtf/templates/dtf/index.html`
  - `twocomms/dtf/templates/dtf/gallery.html`
  - `twocomms/dtf/templates/dtf/effects_lab.html`

Критерии готовности

- визуально совпадает по поведению с референсом (autoplay + ручной контроль)
- без CLS
- mobile не ломает вертикальный scroll

6.2 Multi-step loader для реальной проверки файла

Цель

Показывать именно реальные этапы preflight, а не fixed таймеры:

1. проверка типа/магии
2. проверка размеров и DPI
3. проверка прозрачности/границ
4. риск тонких линий/детализации
5. итог PASS/WARN/FAIL

Что уже есть в backend

В `twocomms/dtf/preflight/engine.py` уже реализовано:

- magic bytes/extension
- file size
- DPI
- alpha channel + bbox + margin
- color mode (RGB/CMYK)
- tiny-detail risk (`PF_TINY_TEXT_RISK`)
- PDF page/media box

Где разрыв

- UI `multi-step-loader.js` сейчас не связан с backend этапами
- в `order` preflight-блок largely статичный

Рекомендованный путь (без Celery)

- оставить sync preflight request-response
- до ответа отображать step-progress честно по фазам клиента (upload/read/send/wait)
- после ответа отображать **реальные checks из JSON** с иконками и объяснениями
- если нужно ощущение “долгой умной проверки”, делать это честно: “анализируем…” + стрим/пошаговая отрисовка уже полученных checks

Где интегрировать

- `twocomms/dtf/static/dtf/js/components/multi-step-loader.js`
- `twocomms/dtf/templates/dtf/order.html`
- `twocomms/dtf/templates/dtf/constructor_app.html`
- (опционально) отдельный endpoint для preflight JSON (сейчас нет dedicated `api/preflight/check/` в `dtf/urls.py`)

6.3 Vanish Input (очистка текста с эффектом)

Цель

При invalid submit/ошибке валидации:

- поле визуально сигнализирует ошибку
- введенный текст очищается через vanish-анимацию
- placeholder циклически подсказывает пример корректного ввода

Что уже есть

- `vanish-input.js`: invalid blur/pulse
- `vanish-input.css`: краткий shake/pulse

Что добавить

1. очередь placeholder'ов + cycle interval
2. onInvalid: animate-out текста и clear value
3. onValid: нормальное поведение без очистки
4. для mobile облегченный режим (без тяжелых particles/canvas)

Где интегрировать

- `twocomms/dtf/static/dtf/js/components/vanish-input.js`
- `twocomms/dtf/static/dtf/css/components/vanish-input.css`
- поля телефонов/емейлов в constructor/order flows

6.4 Floating Dock + нижнее меню + левый тулбар конструктора

Цель

Собрать один coherent паттерн навигации:

- desktop: floating dock с "док-ощущением"
- mobile: bottom dock
- constructor: слева интерактивный toolbar/sidebar (под выбор шагов/инструментов)

Что сейчас конфликтует

В `base.html` одновременно присутствуют:

- `#dtf-fab`
- `[data-floating-dock]`
- `.mobile-dock`

Это визуально и логически конфликтует.

Что сделать

1. унифицировать в единый navigation contract
2. FAB действие “Менеджер” встроить в dock item
3. mobile оставить как responsive режим того же компонента
4. для constructor внедрить отдельный `constructor-sidebar` эффектный режим (expand/collapse), без дублирования глобального dock UX

Где интегрировать

- `twocomms/dtf/templates/dtf/base.html`
- `twocomms/dtf/static/dtf/js/components/floating-dock.js`
- `twocomms/dtf/static/dtf/css/components/floating-dock.css`
- `twocomms/dtf/templates/dtf/constructor_app.html`

6.5 Бегущие карточки для блога (а не отзывы), SEO-safe

Цель

Сделать движущийся блок именно по blog/knowledge контенту, но с сохранением индексации.

Что сейчас

- `infinite-cards` используется в `effects_lab` (демо)
- на главной knowledge-блок сейчас bento-grid статический

Что сделать

1. источник карточек = реальные посты из `knowledge_posts_preview`
2. первый трек = indexable HTML
3. клоны для бесшовности = `aria-hidden="true" inert role="presentation"`
4. pause-on-hover/focus
5. reduced motion fallback: static list

Где интегрировать

- `twocomms/dtf/templates/dtf/index.html` (knowledge section)
- `twocomms/dtf/static/dtf/js/components/effect.infinite-cards.js`
- `twocomms/dtf/static/dtf/css/components/effect.infinite-cards.css`

6.6 “Эффект скорости текста” и дополнительные текстовые эффекты

Цель

Добавить controlled micro-motion в тексты типа “Отправка день в день / скорость / оперативность”, без перегруза.

Рекомендация

- использовать lightweight текстовый jitter/scan только на акцентных словах
- длительность короткая (120–240ms)
- при reduced motion отключать
- SSR-текст всегда читаем до JS

Где

- hero/CTA блоки в `index.html`
- отдельный легкий модуль `effect.text-speed.js` (или расширение `effect.encrypted-text.js`)

---

7. Preflight: что уже покрыто и что нужно добавить

7.1 Уже покрыто в `engine.py`

- формат и magic bytes
- размер файла
- DPI (warn если <300)
- alpha channel presence
- bounding box + margin risk
- color mode (RGB/CMYK)
- tiny detail risk heuristic
- PDF page count и media box

7.2 Что запросили дополнительно (и это нужно)

Ваши обязательные проверки:

- тонкие линии
- белая подкладка / поведение на подложке
- прозрачность
- не выходит ли за края
- соответствие 60 см
- DPI

Реальный статус

- `прозрачность`, `границы`, `DPI` уже есть частично/полностью
- `60 см` пока не валидация rule-level (есть данные размеров, но нет явного pass/fail по 60см)
- `тонкие линии` сейчас косвенно через heuristic, нужен более прозрачный rule и message
- `белая подкладка` как отдельная проверка бизнес-логики отсутствует

Что добавить в engine

1. `PF_WIDTH_60CM_OK/WARN/FAIL`
   - для raster: вычислять физическую ширину через `width_px / dpi * 2.54`
   - для pdf: через `mediabox_pt * 2.54 / 72`
2. `PF_STROKE_THIN_RISK`
   - улучшить алгоритм тонких элементов (mask + morphology) и explainable threshold
3. `PF_WHITE_BACKGROUND_RISK`
   - если фон непрозрачный и почти белый при отсутствии alpha — предупреждать
4. нормализованный UI mapping code→human label (сейчас шаблон может ожидать `label`, а engine дает `code/message`)

---

8. Redis/Celery (“селлари”) и альтернативы

8.1 Факт сейчас

- Celery зависимости есть в коде/requirements
- production broker URL задан
- broker DNS в production не резолвится
- worker-процессы не работают

Значит на текущем контуре **Celery сейчас не operational**.

8.2 Альтернатива без Redis (рекомендуется сейчас)

Вариант A: Sync-first + cron (рекомендуемый для текущего этапа)

- preflight, critical UX checks — sync
- тяжелые не-критичные задачи — cron management commands
- пользователь не зависит от фоновго queue health

Плюсы:

- стабильность на shared/cPanel
- простой дебаг
- предсказуемый UX

Минусы:

- ограниченный real-time async experience

Вариант B: Managed Redis free tier + Celery worker (только при готовой infra)

Возможен после подтверждения:

- DNS/egress до Redis провайдера работает
- есть где держать worker-процесс 24/7 (VPS/контейнер/process manager)

Без этого Celery будет “в коде” но не “в проде”.

Вариант C: Huey/DB-backed queue как переходный

Возможен как временный путь, но на shared тоже рискован без гарантированного daemon process.

8.3 Что делать прямо сейчас

- не блокировать UX реализацию на Celery
- строить preflight и эффекты в sync-mode
- async вернуть после инфраструктурной нормализации (или миграции с shared-хостинга)

---

9. Конкретная карта внедрения (не про “сложность”, а про точность интеграции)

9.1 Wave 1 (must-have ядро)

1. Unified right-bottom nav (Dock/FAB/mobile)
2. Compare parity (drag+hover+autoplay)
3. Real preflight step rendering (без fake таймеров)
4. Vanish clear behavior
5. Knowledge cards as infinite moving blog block (SEO-safe)

9.2 Wave 2 (усиление конструктора)

1. Left interactive constructor toolbar/sidebar
2. Upload dock item with image badge + hover animation on desktop
3. Detailed preflight result cards (каждый check с guidance)
4. Compare-in-preflight preview for WARN quality cases

9.3 Wave 3 (refine + analytics)

1. Event schema + telemetry
2. A/B only where есть трафик и baseline
3. adaptive UX mode newcomer/pro

---

10. Что нужно подготовить coding-агенту перед следующей итерацией

Минимальный пакет:

1. **Component contracts** (JSON/TS-like) для:
   - compare
   - preflight-flow
   - infinite-blog-cards
   - vanish-input
   - unified-dock
2. **Message catalog** для preflight checks (UA/RU/EN)
3. **Feature-flag map** (что включено/выключено в rollout)
4. **Acceptance criteria** по каждому эффекту
5. **Template placement map** (где какой компонент должен жить)

---

11. Что нужно исследовать следующему агенту (Opus 4.6) — обязательный research brief

11.1 Business/behavior блок

1. Для DTF аудитории (новички/про) какие preflight формулировки лучше снижают drop-off?
2. Как лучше показывать WARN так, чтобы не снижать доверие?
3. Что конвертит лучше для главной: moving blog cards vs static knowledge bento?
4. Где граница между “wow” и “шум” в B2B/B2C печатном сервисе?

11.2 Technical/web research блок

1. Лучшие production-паттерны before/after compare для e-commerce print services
2. SEO-safe infinite marquee patterns (SSR-first, decorative clones)
3. Pattern library для multi-step loaders, где шаги отражают реальные backend checks
4. Подходы к легкому vanish effect без тяжелой графики

11.3 Infrastructure блок

1. Когда sync preflight перестает быть достаточным по latency/throughput
2. На каких минимальных условиях Celery реально жизнеспособен (infra checklist)
3. Какие managed Redis/free tiers реально подходят под ожидаемую нагрузку

---

12. Вопросы, которые нужно задать Opus 4.6

1. Сформируй лучший UX-текст для PASS/WARN/FAIL preflight (UA/RU/EN, с tone variants).
2. Дай 2–3 сценария адаптивной логики для новичка и профи, без перегруза интерфейса.
3. Сравни moving blog cards vs static cards по ожидаемому влиянию на trust и SEO.
4. Предложи строгий policy: где autoplay допустим, где только manual interaction.
5. Определи оптимальный motion budget для мобильного трафика DTF-сайта.
6. Сформируй модель риска/выгоды для Celery в текущем окружении и после миграции на VPS.
7. Дай структуру финального prompt для coding-агента, чтобы он сразу писал правильный код без переизобретения архитектуры.
8. Проверь, какие подходы конкурентов (printful/printify/customink/другие) можно переносить без потери конверсии.
9. Дай уточненные критерии “визуально как в Effects.MD” в формате checklist для QA.
10. Предложи план тестов (manual + automation) для эффектов compare/vanish/loader/dock/cards.

---

13. Готовый промпт для Opus 4.6 (вставлять как есть)

```
Ты получаешь технический документ по проекту dtf.twocomms.shop (Django
SSR + Vanilla JS effects).
Твоя задача: провести глубокий research-аудит и вернуть практические
решения, которые можно напрямую перевести в coding-задачи.

Контекст, который нужно считать фактом:
1) В проекте уже есть working effect-layer
(DTF.registerEffect/core.js), но часть эффектов недореализованы.
2) Compare / multi-step loader / vanish input / floating dock /
infinite cards — must-have направления.
3) В production сейчас нет надежного рабочего Celery/Redis контура
(broker DNS не резолвится, worker не найден).
4) Constructor preflight уже sync на backend, но UI и order-flow
нуждаются в доработке и унификации.

Что от тебя нужно:
A. Проведи исследование best practices (UX, SEO, performance, print-
on-demand паттерны) и предложи только применимые для текущего стека
```

```
решения.
B. Оцени, какие идеи из отчета o1/1.md точные, какие нет, и как их
переписать в actionable форму.
C. Сформируй финальный implementation brief для coding-агента:
   - архитектурные принципы
   - contracts компонентов
   - feature flag strategy
   - acceptance criteria
   - roadmap волнами
D. Дай отдельный блок про Celery (селлари):
   - что можно делать без него уже сейчас
   - минимальные условия для возвращения Celery позже
   - альтернативы на случай shared hosting ограничений
E. Сформируй пакет вопросов владельцу проекта, если каких-то данных
объективно не хватает.

Важно:
- Не предлагай полную миграцию на React/Vue как обязательный путь.
- Сохраняй SSR+Vanilla-first подход.
- Любые “wow” эффекты проверяй на SEO, readability, trust и mobile
performance.
- Ответ должен быть максимально конкретным и технически применимым.
```

---

14. Ключевые технические риски и неочевидные места (для повторной проверки)

1. В `constructor_app.html` preflight rendering ожидает `item.label`, а engine checks дают `code/message` — требуется нормализация данных для UI.
2. В `order.html` preflight terminal выглядит как реальный, но сейчас фактически не связан с real checks pipeline.
3. `dtf.js` содержит legacy-функции (`initCompare`, `initTracingBeam`), которые важно либо удалить, либо синхронизировать с новым effect layer.
4. Конфликт навигации в нижнем правом углу может маскировать реальные проблемы UX/конверсии.
5. Feature flags существуют, но не покрывают весь план must-have rollout.

---

15. Источники для внешней сверки (исследовательский минимум)

- Google JS SEO Basics: https://developers.google.com/search/docs/crawling-indexing/javascript/javascript-seo-basics
- MDN `aria-hidden`: https://developer.mozilla.org/en-US/docs/Web/Accessibility/ARIA/Attributes/aria-hidden
- MDN `inert`: https://developer.mozilla.org/en-US/docs/Web/HTML/Reference/Global_attributes/inert
- MDN Vibration API: https://developer.mozilla.org/en-US/docs/Web/API/Vibration_API

- Celery docs (getting started/next steps): https://docs.celeryq.dev/en/stable/getting-started/next-steps.html
- cPanel cron jobs: https://docs.cpanel.net/cpanel/advanced/cron-jobs/
- Huey Django integration: https://huey.readthedocs.io/en/stable/django.html
- Upstash pricing (managed Redis option): https://upstash.com/pricing
- Redis pricing: https://redis.io/pricing/

---

16. Финальный вывод

`Ideas/Audit/o1/1.md` — полезный, но не полностью “ready-to-implement” документ.
После факт-чека:

- стратегическое направление в целом верное;
- ряд формулировок нужно переписать более точно под текущий код и production;
- главная практическая линия: **не ждать Celery**, а внедрять must-have эффекты и real preflight UX уже сейчас на sync-архитектуре;
- следующий Opus-цикл должен работать уже не “с нуля”, а с этим техническим фундаментом и исследовательским брифом.